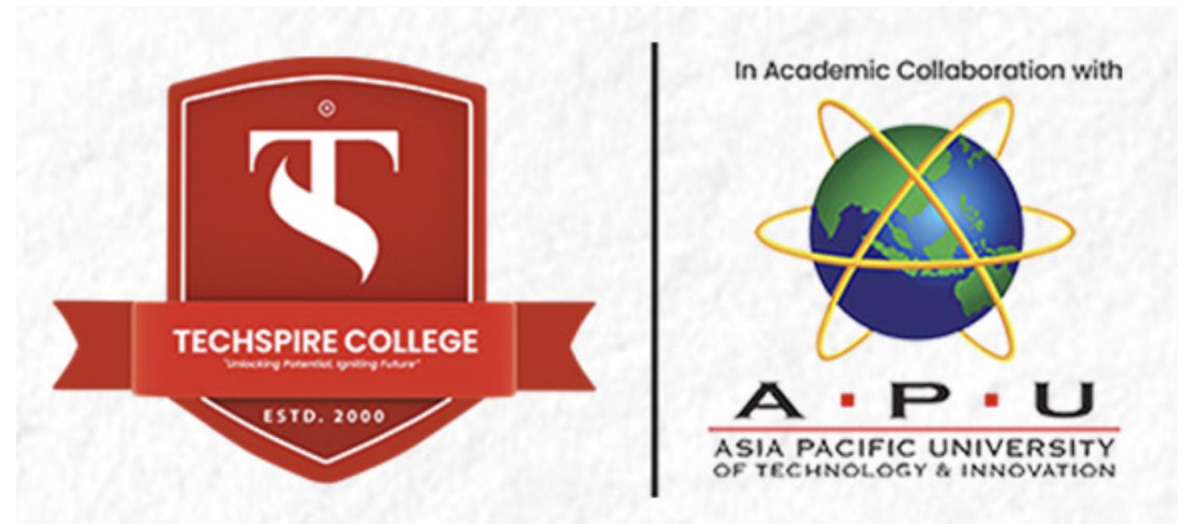


Slide 6: Introduction to Object-Oriented Programming: Classes, Methods, and Key Principles

Ishan Shrestha

# OOP in Java



# What is OOP?

---

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects".

---

Objects represent real-world entities with attributes (data) and behaviors (methods).

---

It allows developers to model and structure programs around objects and their interactions.

# Key Principles of OOP

## Encapsulation:

- Bundling data (fields) and methods (functions) together into a single unit (class).
- **Example:** Hiding internal details of an object and exposing only essential features via access methods (getters/setters).

## Inheritance:

- Enables one class (child) to inherit the properties and behavior of another class (parent).
- Promotes code **reuse** and reduces redundancy.

## Polymorphism:

- "**Many forms**": The ability of objects to behave differently based on their type.
- Achieved through **method overloading** (compile-time) and **method overriding** (runtime).

## Abstraction:

- Hiding implementation details and exposing only essential features.
- Achieved via **abstract classes** and **interfaces**.

# Advantages of OOP

---

**Modularity:** Code is organized into self-contained objects.

---

**Reusability:** Classes and methods can be reused in different parts of the program.

---

**Scalability:** Easy to add new features without altering existing code.

---

**Maintainability:** Changes can be made to individual objects without affecting others.

---

**Real-world Mapping:** Makes it easier to model real-world systems.

# Example

```
Car.java x
1 // Defining the Car class
2 public class Car { no usages
3     // Attributes (fields)
4     private String color; 1 usage
5     private String brand; 4 usages
6     private int speed; 5 usages
7
8     // Constructor
9     public Car(String color, String brand) { no usages
10         this.color = color;
11         this.brand = brand;
12         this.speed = 0; // Default speed is 0
13     }
14
15     // Behaviors (methods)
16     public void start() { no usages
17         System.out.println(brand + " car has started.");
18     }
19
20     public void stop() { no usages
21         speed = 0;
22         System.out.println(brand + " car has stopped.");
23     }
24
25     public void accelerate(int increase) { no usages
26         speed += increase;
27         System.out.println(brand + " car accelerated to " + speed + " km/h.");
28     }
29
30     // Getter for speed (optional)
31     public int getSpeed() { no usages
32         return speed;
33     }
34 }
```

```
Main.java x
1 import java.util.Scanner;
2
3 public class Main {
4     public static void main(String[] args) {
5         Car myCar = new Car( color: "Red", brand: "Toyota");
6         // Demonstrate behaviors
7         myCar.start();
8         myCar.accelerate( increase: 50);
9         myCar.accelerate( increase: 30);
10        myCar.stop();
11    }
12 }
```

```
Toyota car has started.
Toyota car accelerated to 50 km/h.
Toyota car accelerated to 80 km/h.
Toyota car has stopped.
```

# Classes and Objects in OOP

A **Class** is a blueprint for creating objects.

- It defines **attributes** (data) and **methods** (behavior).

An **Object** is an instance of a class.

- It represents a real-world entity and has a state and behavior.

# Class vs Object

---

| Class                             | Object                                             |
|-----------------------------------|----------------------------------------------------|
| Blueprint for objects.            | Instance of a class.                               |
| Does not consume memory.          | Occupies memory.                                   |
| Defines properties and behaviors. | Represents a specific instance with actual values. |

# Key Components of a Class

**Attributes (Fields):** Variables that define the state or properties of an object.

- Example: color, brand, speed in a Car class.

**Methods:** Functions inside a class that define its behavior.

- Example: start(), stop(), accelerate() in a Car class.

# Understanding Methods in Java

A **method** is a block of code that performs a specific task.

Methods are used to:

**Reuse code** to avoid duplication.

**Modularize programs** for better readability and maintenance.

# Method Signature

```
public int addNumbers(int a, int b) {  
    return a + b;  
}
```

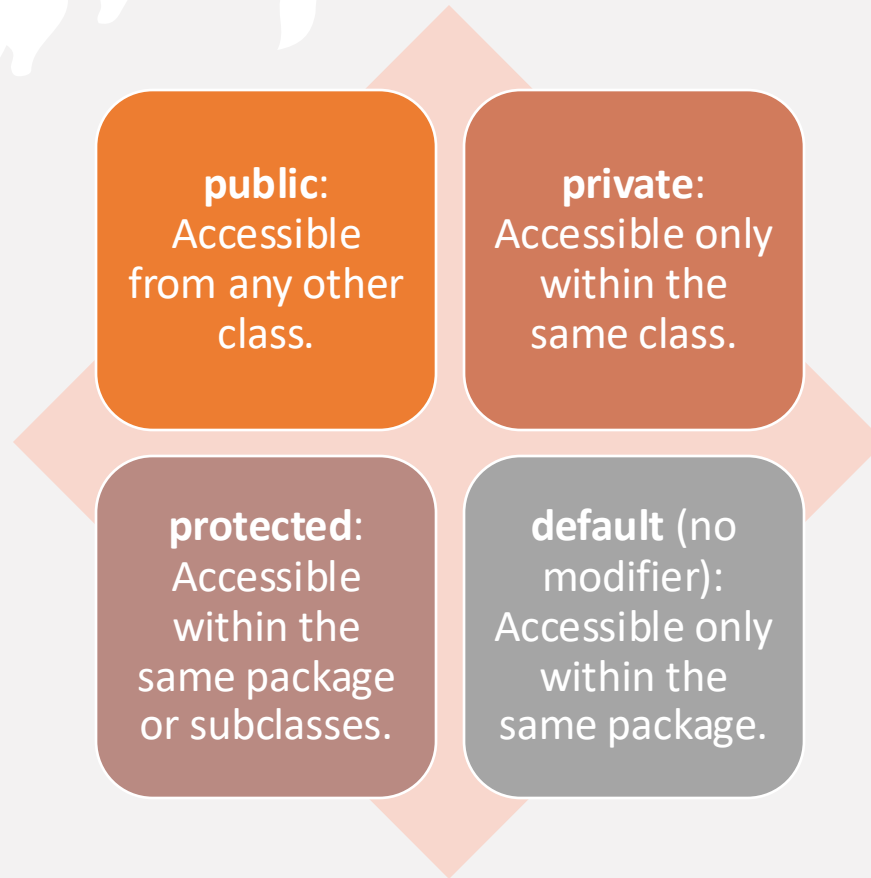
- The **method signature** defines the structure of a method and includes:
  - **Access Modifier:** Determines the visibility of the method.
  - **Return Type:** Specifies the type of value the method returns (void if it returns nothing).
  - **Method Name:** A unique identifier that represents the method.
  - **Parameters:** Input values the method can accept, enclosed in parentheses.

# Components of a Method

---

| Component        | Description                                                     |
|------------------|-----------------------------------------------------------------|
| Access Modifier  | Controls visibility (public, private, protected, default).      |
| Return Type      | Type of value returned (e.g., int, String, void).               |
| Method Name      | Identifier used to call the method.                             |
| Parameters       | Inputs for the method, written in parentheses.                  |
| Body             | Contains the logic or task the method performs, enclosed in {}. |
| Return Statement | Specifies the value to return (not needed for void methods).    |

# Access Modifiers in Methods



# Constructors in Java

A **constructor** is a special method used to initialize objects.

It is called automatically when an object is created.

## Key Points:

- Has the same name as the class.
- Does not have a return type (not even void)

# Types of Constructors

## Default Constructor:

- No arguments provided.
- Automatically added by Java if no other constructor is defined.

## Parameterized Constructor:

- Accepts parameters to initialize attributes during object creation.

## Constructor Overloading:

- Multiple constructors with different parameter lists in the same class.

# Practice Questions

---

Create a class Book with attributes title, author, and price. Write a method displayDetails() that prints the details of the book. Create a Book object in the main method, set the values for the attributes, and call the displayDetails() method to display the information.

---

Create a class Rectangle with attributes length and width. Write a method calculateArea() that returns the area of the rectangle. Create a Rectangle object in the main method, set the length and width, and print the area using the calculateArea() method.